

Linguahe: Philippine Language Identification System

Kirk Henrich Cambel Gamo, Clarence Anthony Bolivar, Jullian Bilan, Jan Floyd Vallota

College of Information and Communications Technology

West Visayas State University

Luna St., La Paz, Iloilo City 5000

Abstract—Linguahe is a web-based language identification system focused on Philippine languages. It classifies text into Tagalog, Cebuano, Hiligaynon, or Ilocano using a character-level CNN and a preprocessing pipeline designed for low-resource language variation. The system targets real-world inputs such as news headlines and short user messages, where spelling variation and shared vocabulary are common. This document summarizes the technical aspects, functional requirements, and roadmap of the developed system.

I. OVERVIEW AND INTRODUCTION

A. App Name

Linguahe

B. Purpose

The Philippines is a multilingual country with many low-resource languages. Linguahe addresses the lack of reliable language identification tools for regional Philippine languages, especially Hiligaynon, by providing a web-based classifier that supports Tagalog, Cebuano, Hiligaynon, and Ilocano. The system is designed as a lightweight, local deployment that can be embedded in downstream applications such as content routing, localization pipelines, and language-specific NLP workflows.

C. Supported Languages

TABLE I
SUPPORTED LANGUAGES

Language	Code
Tagalog	tagalog
Cebuano	cebuano
Hiligaynon	hiligaynon
Ilocano	ilocano

D. Technology Stack (with Versions)

The system is implemented using a modern Python ML stack and a lightweight web frontend. All components are locally deployable to meet classroom and offline constraints, with model artifacts stored in HDF5 and joblib formats for portability. The backend uses FastAPI with TensorFlow/Keras for model

TABLE II
CORE TECHNOLOGY STACK

Component	Version
Python	3.13
FastAPI	0.136
TensorFlow	2.21
Keras	3.14
scikit-learn	1.3+
pandas	2.0+
numpy	2.0+
joblib	1.3+

inference and scikit-learn utilities for baseline training and data handling. Data ingestion and preparation rely on pandas, numpy, BeautifulSoup4, datasets, and requests. The frontend is implemented with HTML5, CSS3, and vanilla JavaScript (ES6).

II. FUNCTIONAL REQUIREMENTS

This section outlines the functional scope implemented in the prototype, including user-facing features, preprocessing logic, and API behaviors.

A. Core Features with Descriptions

Linguahe accepts user input and classifies it into one of four supported languages using a softmax output. The preprocessing pipeline normalizes Unicode, removes noise (URLs, mentions, hashtags), strips digits and excessive punctuation, and

standardizes Hiligaynon spelling variants. The system exposes a REST endpoint (`/predict`) that returns the predicted label and confidence scores, which the single-page web interface renders as a result card and bar chart.

B. Libraries/Packages

Training and inference require FastAPI, TensorFlow/Keras, scikit-learn, pandas, numpy, and joblib. Data collection utilities include beautifulsoup4, datasets, and requests.

C. Dataset Summary

The training dataset aggregates Philippine language text from curated sources and cleans it into a unified format.

TABLE III
DATASET DISTRIBUTION

Language	Samples
Cebuano	46347
Tagalog	45831
Hiligaynon	7264
Ilocano	5998
Total	105440

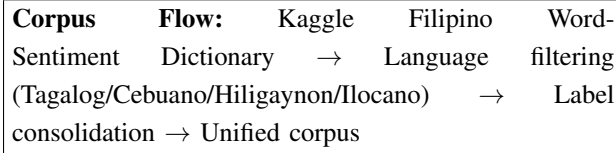


Fig. 1. Training corpus assembly pipeline.

D. Training Corpora Selection

To ensure coverage of regional Philippine languages, the training corpus was selected from the Kaggle “Filipino Word-Sentiment Dictionary” dataset (<https://www.kaggle.com/datasets/abeperez/filipino-wordsentiment-dictionary>). Cebuano, Tagalog, Hiligaynon, and Ilocano texts were consolidated into a unified corpus with consistent labeling. The focus was on common lexical patterns and everyday vocabulary to maximize generalization to short user inputs and headlines.

E. Preprocessing Rules

The pipeline lowercases and normalizes Unicode (NFC), removes diacritics, strips URLs/mentions/hashtags, removes digits and non-essential punctuation, normalizes whitespace, and applies Hiligaynon spelling normalization.

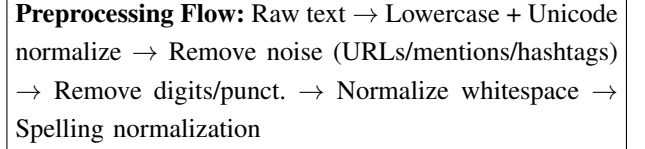


Fig. 2. Text preprocessing pipeline.

F. NLP Task and Implementation

The core NLP task is multi-class language identification. Given an input string, the system predicts one of four languages using a character-level representation, which is robust to misspellings and out-of-vocabulary words. Implementation is performed by character tokenization, sequence padding to a fixed length, and inference through a CNN classifier that outputs softmax probabilities across classes.

G. Model Architecture

TABLE IV
CHARACTER-LEVEL CNN ARCHITECTURE

Layer	Output Shape	Notes
Embedding	(50, 64)	Char embeddings
Conv1D (k=3, 128)	(48, 128)	ReLU
MaxPool1D (2)	(24, 128)	Downsample
Conv1D (k=3, 128)	(22, 128)	ReLU
GlobalMaxPool1D	(128)	Global pooling
Dense	(64)	ReLU
Dropout	(64)	p=0.5
Dense	(4)	Softmax

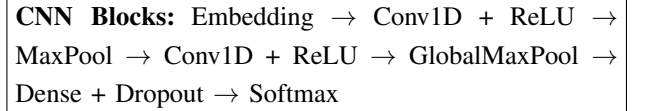


Fig. 3. High-level CNN architecture blocks.

H. Model Training and Development

Model development begins with preprocessing and character tokenization. Labels are encoded using a label encoder, and sequences are padded to a fixed length. The CNN is trained with early stopping on validation accuracy to avoid overfitting, and artifacts (model, tokenizer, encoder) are saved for deployment. A baseline TF-IDF + MultinomialNB model is also trained for comparison.

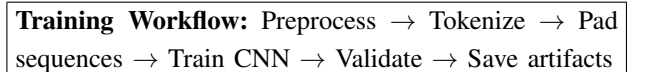


Fig. 4. Model training and artifact saving workflow.

I. Training Setup

Training uses an 80/20 train-test split with stratification. The model uses a 50-character max sequence length, Adam optimizer with sparse categorical cross-entropy, batch size 64, and 10 epochs with early stopping (patience=3).

J. Evaluation Metrics

The system reports accuracy and per-class precision, recall, and F1-score. The current CNN achieves approximately 70% accuracy on the held-out test set. Confusion matrix analysis is used to inspect error patterns among closely related languages.

Evaluation Flow: Split data → Train → Predict on test set → Compute metrics + confusion matrix

Fig. 5. Evaluation and reporting workflow.

K. Model Evaluation and Analysis

Evaluation uses a stratified train-test split to preserve class proportions. Performance is assessed with classification reports and confusion matrices to identify confusion between Cebuano and Hiligaynon, which share orthographic patterns. Error inspection informs preprocessing adjustments and motivates future improvements such as multi-kernel CNNs and confidence calibration.

L. API Specification

Endpoint: POST /predict

Request JSON: {"text": "<input string>"}

Response JSON: {"language": "<label>", "confidence": {...}}

M. Reproducibility and Environment

Training uses a fixed random seed (random_state=42) for data splitting. Experiments were executed on a CPU-based local machine with a Python virtual environment and pinned package versions.

N. Ethical Considerations

Model performance can be biased toward high-resource classes due to class imbalance. Misclassification may affect downstream applications such as localization or content routing, so confidence scores are exposed to aid human oversight.

System Flow: Input text → Preprocess → Char Tokenize → Pad → CNN Inference → Predicted Language + Confidence

Fig. 6. High-level inference workflow for Linguahe.

O. Technical Workflow

Input Flow: User text is preprocessed, tokenized at the character level, padded to a fixed length, and passed to the CNN for inference. The predicted label and confidence scores are returned as JSON to the frontend.

Modeling Details: The CNN uses an embedding layer, two Conv1D layers with ReLU activations, max pooling, and a dense classifier head. Training uses early stopping on validation accuracy to avoid overfitting.

III. FUTURE ENHANCEMENTS AND ROADMAP

This roadmap focuses on improving robustness for short inputs, extending language coverage, and increasing accuracy using stronger architectures.

A. Known Limitations

Cebuano and Hiligaynon share many orthographic forms, which increases confusion for short inputs. Very short phrases reduce classification reliability, and informal or code-mixed text can shift the input distribution. The fixed 50-character context window may also truncate longer inputs.

B. Planned Features and Improvements

Planned work includes token-level language identification for mixed-language text, expanded coverage (Kinaray-a, Akeanon), and stronger architectures such as multi-kernel CNNs or transformer-based models. Evaluation will be strengthened using a fixed holdout set and deeper error analysis, with confidence calibration via temperature scaling or Platt scaling. Additional improvements include a confidence threshold to label low-certainty predictions as uncertain, UI enhancements such as batch input and CSV upload, lightweight model compression for faster inference, and an active-learning loop for periodic retraining from user feedback.